

第一次上机实验

第一题

求可行的方案数，题面说 5，1，1 和 1，5，1 算一种情况，其实可以转化为求单调不增序列，序列之和为 n 这样序列的个数。dfs 维护父状态的个数。

时间复杂度 $O(n!)$

代码：

```

#include <iostream>
using namespace std;
#define int long long
#define inf 0x3f3f3f3f3f3f3f3f
#define pii pair<int, int>
#define endl '\n'
const int N=1e6+10;
const int N1=1e3+10;
const int mod=1e9+7;
int ans=0;
int n,k;
void dfs(int num,int pos,int mx){//剩几个苹果剩几个盘子当前盘子最多能放多少个
    // cout<<num<<" "<<pos<<" "<<mx<<endl;
    if(pos>k) return;
    if(num==0){
        ans++;
        return;
    }
    for(int i=num;i>=0;i--){//拿了num个
        if(i>mx) continue;
        dfs(num-i,pos+1,i);//最多不会超过当前num
    }
}
void Atomic_AC_machine(){
    cin>>n>>k;
    ans=0;
    dfs(n,0,n);
    cout<<ans<<endl;
}
signed main(){
    ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);
    int t=1;
    cin>>t;
    while(t--){
        Atomic_AC_machine();
    }
    return 0;
}

```

第二题

观察到每一个图形可以分成九宫格中的五个子图形，这样只需要一直递归直到子图形大小为 1，放置字符'X'就可以了。

时间复杂度 $O(5^n)$

代码：

```

#include <bits/stdc++.h>
using namespace std;
#define int long long
#define inf 0x3f3f3f3f3f3f3f3f
#define pii pair<int, int>
#define endl '\n'
const int N=1e6+10;
const int N1=1e3+10;
const int mod=1e9+7;
char ans[1000][1000];
void dfs(int x1,int x2,int y1,int y2){
    // cout<<x1<<" "<<x2<<" "<<y1<<" "<<y2<<endl;
    if(x1==x2&&y1==y2){
        ans[x1][y1]='X';
        return;
    }
    int len=(x2-x1+1)/3;
    dfs(x1,x1+len,y1,y1+len);
    dfs(x1+2*len,x1+3*len,y1,y1+len);

    dfs(x1+len,x1+2*len,y1+len,y1+2*len);

    dfs(x1,x1+len,y1+2*len,y1+3*len);
    dfs(x1+2*len,x1+3*len,y1+2*len,y1+3*len);
}
signed main(){
    // ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);
    int mx=pow(3,7-1);
    for(int i=1;i<=mx;i++){
        for(int j=1;j<=mx;j++){
            ans[i][j]=' ';
        }
    }
    // cout<<mx<<endl;
    dfs(1,mx,1,mx);
    int t;
    while(cin>>t){
        if(t==-1) return 0;
        for(int i=1;i<=pow(3,t-1);i++){
            for(int j=1;j<=pow(3,t-1);j++){
                cout<<ans[i][j];
            }cout<<endl;
        }
    }
}

```

```
        cout<<"-"<<endl;
    }
    return 0;
}
```

第三题

很典的三角形 dp,转移方程 $dp[i][j] = \min(dp[i-1][j], dp[i-1][j-1])$ 。

时间复杂度 $O(n^2)$

```

#include <iostream>
using namespace std;
#define int long long
#define inf 0x3f3f3f3f3f3f3f3f
#define pii pair<int, int>
#define endl '\n'
const int N=1e6+10;
const int N1=1e3+10;
const int mod=1e9+7;
int dp[N1][N1];
int a[N1][N1];
void Atomic_AC_machine(){
    int n;cin>>n;
    for(int i=1;i<=n;i++){
        for(int j=1;j<=i;j++){
            cin>>a[i][j];
        }
    }
    int ans=0;
    for(int i=1;i<=n;i++){
        for(int j=1;j<=i;j++){
            dp[i][j]=a[i][j]+max(dp[i-1][j],dp[i-1][j-1]);
            ans=max(ans,dp[i][j]);
        }
    }
    cout<<ans<<endl;
}
signed main(){
    ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);
    int t=1;
    // cin>>t;
    while(t--){
        Atomic_AC_machine();
    }
    return 0;
}

```

第四题

求最长递增子序列, $dp[i]$ 从前面 $dp[j]$ 最大的且 $a[i] > a[j]$ 的位置转移。
 因为数据是 1000 可以暴力 $O(n^2)$, 其实可以二分/树状数组做到 $O(n \log(n))$ 。

代码:

```

#include <iostream>
#include <queue>
// #incldue <algorithm>
using namespace std;
#define int long long
#define inf 0x3f3f3f3f3f3f3f3f
#define pii pair<int, int>
#define endl '\n'
const int N=1e6+10;
const int N1=1e3+10;
const int mod=1e9+7;
int a[N],dp[N];
void Atomic_AC_machine(){
    int n;cin>>n;
    for(int i=1;i<=n;i++) cin>>a[i];
    for(int i=1;i<=n;i++){
        int mx=0;
        for(int j=1;j<=i-1;j++){
            if(a[i]>a[j]){
                mx=max(mx,dp[j]);
            }
        }
        // cout<<i<<" "<<mx<<endl;
        dp[i]=mx+1;
        // cout<<dp[i]<<" ";
    }
    // cout<<endl;
    int ans=0;
    for(int i=1;i<=n;i++){
        ans=max(ans,dp[i]);
    }
    cout<<ans<<endl;
}
signed main(){
    ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);
    int t=1;
    // cin>>t;
    while(t--){
        Atomic_AC_machine();
    }
    return 0;
}

```


第五题

区间dp

$dp[i][j]$ 表示区间 i 到 j 变成回文串的花费。

- case 1: $s[i] == s[j]$ 这样只需要保证从 $i + 1$ 到 $j - 1$ 是回文串就可以, 方程 $dp[l][r] = dp[l + 1][r - 1]$
- case 2: $s[i] \neq s[j]$ 这样只需要保证从 $i + 1$ 到 j (在左端插入 $s[j]$ 或右端删除 $s[i]$) 或者 i 到 $j - 1$ 是回文串 (在左端插入 $s[r]$ 或右端删除 $s[r]$)。
- 方程 $dp[l][r] = \min(dp[l + 1][r] + cost[s[l] - 'a'], dp[l][r - 1] + cost[s[r] - 'a'])$ 。

时间复杂度 $O(n^2)$

代码:

```

#include <bits/stdc++.h>
using namespace std;
#define int long long
#define inf 0x3f3f3f3f3f3f3f3f
#define pii pair<int, int>
#define endl '\n'
const int N=1e6+10;
const int N1=3e3+10;
const int mod=1e9+7;
string s;
int n,m;
int cost[26];
int dp[N1][N1];
void Atomic_AC_machine(){
    cin>>n>>m>>s;
    s=" "+s;
    for(int i=1;i<=n;i++){
        char ch;
        int del,add;
        cin>>ch>>del>>add;
        cost[ch-'a']=min(del,add);
    }
    for(int len=1;len<=m;len++){
        for(int l=1;l+len-1<=m;l++){
            int r=l+len-1;
            if(len==1){
                dp[l][r]=0;
            }else{
                if(s[l]!=s[r]) dp[l][r]=min(dp[l+1][r]+cost[s[l]-'a'],dp[l][r-1]+cost[s[r]-'a']);
                else{
                    dp[l][r]=dp[l+1][r-1];
                }
            }
        }
    }
    // for(int i=1;i<=n;i++){
    //     for(int j=i;j<=n;j++){
    //         cout<<i<<" "<<j<<" "<<dp[i][j]<<" "<<endl;
    //     }
    // }
    cout<<dp[1][m]<<endl;
}
signed main(){

```

```
ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);  
int t=1;  
// cin>>t;  
while(t--){  
    Atomic_AC_machine();  
}  
return 0;  
}
```

第二次上机实验

第一题：

- 最近点对，分治法时间复杂度 $O(n\log n)$

```

#include <iostream>
#include <algorithm>
#include <cmath>
#include <iomanip>
using namespace std;
#define ll long long
const int inf=0x3f3f3f3f;
const ll infll=0x3f3f3f3f3f3f3f3f;
#define int long long
#define pii pair <int,int>
#define ld long double
#define endl "\n"
const int N=200050;
struct node {
    int x,y,type;
}A[N];
int cmp(node x,node y){
    if (x.x!=y.x)return x.x<y.x;
    return x.y<y.y;
}
int cmp2(node x,node y){
    return x.y<y.y;
}

int abss(int x){
    if (x<0)return -x;
    return x;
}
node B[N];
double mindis;
double res;
void upd(node a, node b) {
    double dis=sqrt(1.0*(a.x-b.x)*(a.x-b.x)+(a.y-b.y)*(a.y-b.y));
    if (dis<res&&a.type!=b.type) res=dis;
    if (dis<mindis)mindis=dis;
}
signed main (){
    ios::sync_with_stdio(false);
    cin.tie(0);cout.tie(0);
    int t;
    cin >>t;
    while (t--){
        int n;

```

```

cin >>n;
for (int i=1;i<=n;i++){
    cin >>A[i].x>>A[i].y;
    A[i].type=0;
}
for (int i=1;i<=n;i++){
    cin >>A[i+n].x>>A[i+n].y;
    A[i].type=1;
}
sort (A+1,A+1+n*2,cmp);
node pre1,pre0;
int flag1=0,flag0=0;
mindis = infll;
res=infll;
for (int i=1;i<=n*2;i++){
    if (A[i].type==1){
        flag1=1;
        pre1=A[i];
        if (flag0)upd(A[i],pre0);
    }else{
        flag0=1;
        pre0=A[i];
        if (flag1)upd(A[i],pre1);
    }
}
cout <<fixed<<setprecision(3)<<res<<endl;

}
}

```

第二题

- 求逆序对，开树状数组，时间复杂度 $O(n\log n)$

```

#include <iostream>
#include <algorithm>
#define int long long
using namespace std;
struct node{
    int val,pos;
}a[510000];
int s[510000];
int n;
int lowbit(int x){
    return x&-x;
}
void change(int x,int k)
{
    while(x<=n) s[x]+=k,x+=lowbit(x);
}
bool cmp(node x,node y){
    if(x.val==y.val){
        return x.pos>y.pos;
    }else return x.val>y.val;
}
int query(int x){//查询区间和
    int t=0;
    while(x) t+=s[x],x-=lowbit(x);
    return t;
}
signed main(){
    int t;cin>>t;
    int id=0;
    while(t--){
        cin>>n;
        for(int i=1;i<=n;i++){
            s[i]=0;
        }
        for(int i=1;i<=n;i++){
            cin>>a[i].val;a[i].pos=i;
        }
        sort(a+1,a+n+1,cmp);
        int sum=0;
        for(int i=1;i<=n;i++){
            change(a[i].pos,1);
            sum+=query(a[i].pos-1);
        }
    }
}

```

```
        cout<<"Scenario #"<<id<<":"<<endl;
        cout<<sum<<endl<<endl;
    }

    return 0;
}
```

第三题

- 放小木棍，剪枝
 - 避免在同一层递归中选择相同的元素值，防止重复计算。
 - 如果找到解，则立即返回 true，不再继续搜索。
 - 如果当前组的累积长度为0，意味着正在尝试构建一个新的组，而此时如果添加某个元素后正好等于目标长度 res，则直接返回 false。
 -

```

#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;
#define int long long
#define inf 0x3f3f3f3f3f3f3f3f
#define pii pair<int, int>
#define endl '\n'
const int N=1e6+10;
const int N1=1e3+10;
const int mod=1e9+7;
int stc[N];
int sum=0;
int res=inf;
int vis[N];
bool dfs(int len,int cntt,int pos,int des,int n){//目前拼了多长，拼了多少个，遍历最左端位置，一根有
    // cout<<len<<" "<<cntt<<" "<<pos<<" "<<des<<endl;
    // if(pos>=n) return -inf;
    if(cntt>=sum/des){
        return true;
    }
    if(len==des){
        return dfs(0,cntt+1,1,des,n);
    }
    int lst=0;
    for(int i=pos;i<=n;i++){
        if(!vis[i]&&len+stc[i]<=des&&lst!=stc[i]){
            lst=stc[i];
            vis[i]=1;
            if(dfs(len+stc[i],cntt,pos+1,des,n)) return true;
            vis[i]=0;
            if(len==0||stc[i]+len==res) return false;
        }
    }
    // return false;
}

void Atomic_AC_machine(int n){
    res=inf;
    sum=0;
    for(int i=1;i<=n;i++){
        cin>>stc[i];
        sum+=stc[i];
    }
}

```



```

sort(stc+1,stc+1+n);
reverse(stc+1,stc+1+n);
// for(int i=1;i<=n;i++){
//     cout<<stc[i]<<" ";
// }
// cout<<endl;
vector<int> fac;
for(int i=stc[1];i<=sum;i++){
    if(sum%i==0){
        fac.push_back(i);
        // cout<<i<<endl;
    }
}
for(int i=0;i<fac.size();i++){
    // cout<<i<<endl;
    for(int i=1;i<=n;i++) vis[i]=0;
    if(dfs(0,0,1,fac[i],n)){
        cout<<fac[i]<<endl;
        return;
    }
}
}
}
signed main(){
    // ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);
    int n;
    while(cin>>n){
        for(int i=1;i<=n;i++) vis[i]=0;
        if(n==0) return 0;
        Atomic_AC_machine(n);
    }
    // return 0;
}

```

第五题

- 问一堆小木棍能不能拼成正方形，搜索
 - 如果成功构建了足够的组，则不再继续搜索。
 - 优化起始点选择：

每次从上次选择的位置 last 开始遍历，而不是每次都从头开始，确保每次递归都从上次选择的位置继续，避免重复计算。

- 长度检查:

只考虑那些未被访问过且加入当前组后不会超过目标长度 `cnt` 的棒子

```

#include <iostream>
#include <cstdio>
#include <algorithm>
#include <cstring>
using namespace std;
int n,m,cnt,sum;
bool vis[101];
int a[101];
bool dfs(int stick,int len,int last){
    if(stick>3) return true;
    if(len==cnt) return dfs(stick+1,0,1);
    for(int i=last;i<=m;i++){
        if(!vis[i]&&a[i]+len<=cnt){
            vis[i]=1;
            if(dfs(stick,len+a[i],i)) return true;
            vis[i]=0;
        }
    }
    return false;
}
int main(){
    cin>>n;
    while(n--){
        sum=0;
        cin>>m;
        for(int i=1;i<=m;i++){
            cin>>a[i];
            sum+=a[i];
        }
        sort(a+1,a+1+m);
        reverse(a+1,a+1+m);
        if(sum%4!=0){
            cout<<"no"<<endl;
            continue;
        }
        cnt=sum/4;
        memset(vis,0,sizeof(vis));
        if(dfs(1,0,1)) cout<<"yes"<<endl;
        else cout<<"no"<<endl;
    }
}

```

第三次上机实验：

第一题

- 对于两个独立的任务，能推出，i在j前面的条件是 $\min(a_i, b_j) < \min(a_j, b_i)$
- 贪心算法排序即可，时间复杂度 $O(n\log n)$ 。

```

#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;
#define int long long
#define inf 0x3f3f3f3f3f3f3f3f
#define pii pair<int, int>
#define endl '\n'
const int N=1e6+10;
const int N1=1e3+10;
const int mod=1e9+7;
struct node{
    int x,y;
};
bool cmp(node a,node b){
    return min(a.x,b.y)<min(a.y,b.x);
}
signed main(){
    ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);
    int n=1;
    while(cin>>n&& n){
        vector<node> a(n);
        for(int i=0;i<n;i++){
            cin>>a[i].x>>a[i].y;
        }
        sort(a.begin(),a.end(),cmp);
        int cost1=0,cost2=0;
        for(int i=0;i<n;i++){
            cost1+=a[i].x;
            cost2=max(cost1+a[i].y,cost2+a[i].y);
        }
        cout<<cost2<<endl;
    }
    return 0;
}

```

第二题

- 求k短路，先用单源最短路跑出估算函数，然后跑A*算法
- 当一个点在队列中出现了k次，返回这个点的估算函数+当前走的距离。

```

#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <iostream>
#include <algorithm>
#include <queue>
#define inf 1000000000
using namespace std;
int n,m,first[1010]={0},qfirst[1010]={0},tot=0,S,T,k;
int h[1010];
bool vis[1010];
struct edge{int v,w;};
vector<edge> e[100010],qe[100010];
struct state
{
    int u,g,h;
    bool operator < (state a) const
    {
        return a.g+a.h<g+h;
    }
};

void insert(int a,int b,int c)
{
    e[a].push_back(edge{b,c});
    qe[b].push_back(edge{a,c});
}

void spfa() //SPFA跑反图，用dijkstra也可以
{
    memset(vis,0,sizeof(vis));
    for(int i=1;i<=n;i++) h[i]=inf;
    queue<int> Q;
    Q.push(T);h[T]=0;
    while(!Q.empty())
    {
        int u=Q.front();
        Q.pop();
        for(int i=0;i<qe[u].size();i++)
        {
            int j=qe[u][i].v;
            if (h[u]+qe[u][i].w<h[j])
            {

```

```

        h[j]=h[u]+qe[u][i].w;
        if (!vis[j]) {Q.push(j);vis[j]=1;}
    }
}
vis[u]=0;
}
}

```

```

int AStar() //A*
{
    priority_queue<state> fq;
    state now,next;
    now.u=S,now.g=0,now.h=h[S];
    fq.push(now);
    int cnt[1010]={0};
    while(!fq.empty())
    {
        now=fq.top();
        fq.pop();
        cnt[now.u]++;
        if(cnt[now.u]==k) return now.g+now.h;
        for(int i=0;i<e[now.u].size();i++)
        {
            next.u=e[now.u][i].v;
            next.g=now.g+e[now.u][i].w;
            next.h=h[e[now.u][i].v];
            fq.push(next);
        }
    }
    return -1;
}

```

```

int main()
{
    scanf("%d%d",&n,&m);
    for(int i=1;i<=m;i++)
    {
        int s,t,c;
        scanf("%d%d%d",&s,&t,&c);
        insert(s,t,c);
    }
    scanf("%d%d%d",&S,&T,&k);
    if (S==T) k++; //特殊情况，S=T
}

```

```
    spfa();

    printf("%d",AStar());

    return 0;
}
```

第三题

- 补图上 Tarjan 求点双，对于每一个点双二分图染色判断是否有奇环，不在任何一个有奇环的点双上的点都无法参加会议。不知道为什么。
- 时间复杂度 $O(V + E)$


```

#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>
using namespace std;
#define ll long long
const int inf=0x3f3f3f3f;
const ll infll=0x3f3f3f3f3f3f3f3f;
//#define int long long
#define pii pair <int,int>
#define ld long double
#define endl "\n"
const int N=1010;

vector <int> e[N];
int dfn[N],low[N],dfncnt=0;
int vis[N];
int cut[N],res=0; //ans[i]=1 -> 点i是割点
int sta[N],top,dcccnt;
int n,m;
int ed[N][N];
vector <int> dcc[N];
int root;
int ans[N];
struct Dsu{
    int f[2*N],color[N];
    void init(int n){
        for(int i=1;i<=n;i++){
            f[i]=i;
        }
    }
    int find(int x){
        if(f[x]!=x) return f[x]=find(f[x]);
        return f[x];
    }

    void join(int x,int y){
        int fx=find(x);
        int fy=find(y);
        if(fx!=fy){
            f[fy]=fx;
        }
    }
}

```

```

};
Dsu dsu;
bool has_odd_cycle(int p) {
    dsu.init(2*n);
    for(int j=0;j<dcc[p].size();j++){
        int u=dcc[p][j];
        for(int i=0;i<e[u].size();i++){
            int v=e[u][i];
            if(u==v) continue;
            if (dsu.find(u) == dsu.find(v)) {
                return true; // 存在奇环
            }else{
                dsu.join(u,v+n);
                dsu.join(v,u+n);
            }
        }
    }
    return false;
}

void Tarjan(int u,int fa){
    vis[u]=1;
    low[u]=dfn[u]=++dfncnt;sta[++top]=u;
    if (u==root&&e[u].size()==0){
        dcc[++dcccnt].push_back(u);
        return;
    }
    int ccnt=0;
    int len=e[u].size();
    for (int i=0;i<len;i++){
        int v=e[u][i];
        if(v==fa)continue;
        if(!vis[v]){
            Tarjan(v,u);
            low[u]=min(low[u],low[v]);
            if (low[v]>=dfn[u]){
                if (++ccnt>1||u!=root)cut[u]=1;
                dcccnt++;
                do dcc[dcccnt].push_back(sta[top--]);
                while (sta[top+1]!=v);
                dcc[dcccnt].push_back(u);
            }
        }
        else low[u]=min(low[u],dfn[v]);
    }
}

```

```

    }
}

```

```

signed main (){
    ios::sync_with_stdio(false);
    cin.tie(0);cout.tie(0);

    while(cin >>n>>m&& n+m!=0){
        for(int i=1;i<=n;i++){
            ans[i]=dfn[i]=low[i]=vis[i]=cut[i]=sta[i]=0;
            e[i].clear();
        }
        for(int i=1;i<=dcccnt;i++){
            dcc[i].clear();
        }
        for(int i=1;i<=n;i++){
            for(int j=1;j<=n;j++){
                ed[i][j]=0;
            }
        }
        res=top=dcccnt=dfncnt=0;
        for (int i=0;i<m;i++){
            int u,v;
            cin >>u>>v;
            ed[u][v]=ed[v][u]=1;
        }
        for(int i=1;i<=n;i++){
            for(int j=i;j<=n;j++){
                if(i==j) continue;
                if(ed[i][j]==0){
                    e[i].push_back(j);
                    e[j].push_back(i);
                }
            }
        }
        dsu.init(n);
        for (int i=1;i<=n;i++){
            if(!vis[i]){
                root=i;Tarjan(i,i);
            }
        }
        for(int i=1;i<=dcccnt;i++){

```

```

        if(has_odd_cycle(i)){
            for(int j=0;j<dcc[i].size();j++){
                ans[dcc[i][j]]=1;
            }
        }
    }
    cout<<n-accumulate(ans+1,ans+1+n,0)<<endl;
}
}

```

第四题

- 多源最短路，松弛的时候取两条边最大值松弛就行了
- 时间复杂度 $O(n^3)$

```

#include <iostream>
#include <cmath>
#include <iomanip>
using namespace std;
const int N1=210;
double d[N1][N1];
pair<double,double> p[N1];
double dis(int i,int j){
    return sqrt((p[i].first-p[j].first)*(p[i].first-p[j].first)+(p[i].second-p[j].second)*(p[i].second-p[j].second));
}
int main(){
    int n;
    int cnt=0;
    while(cin>>n&& n){
        for(int i=1;i<=n;i++){
            cin>>p[i].first>>p[i].second;
        }
        for(int i=1;i<=n;i++){
            for(int j=1;j<=n;j++){
                d[i][j]=dis(i,j);
            }
        }
        for(int k=1;k<=n;k++){
            for(int i=1;i<=n;i++){
                for(int j=1;j<=n;j++){
                    d[i][j]=min(max(d[i][k],d[k][j]),d[i][j]);
                }
            }
        }
        cout<<"Scenario #"<<cnt<<endl;
        cout<<fixed<<setprecision(3)<<"Frog Distance = "<<d[1][2]<<endl<<endl;
        cnt++;
    }
}

```

第五题

- 每次插入一条边就跑一遍拓扑排序
- 当没有入度为零的点的时候，显然条件有矛盾。同时，当出现矛盾时，有些点的入度永远不会减到零，这就意味着这些点永远入不了队。

- 如果最开始入度为零的点有不只一个，那么条件一定不足，因为这两个点的顺序无法确定。同时，如果每次取队首时，有不只一个点入队了，那么条件一定不足，因为这两个点都只有队首连向的边。所以这两个点无法确定顺序。
- 如果同时满足条件矛盾与条件不足，这时应该判定为条件矛盾。
- 时间复杂度 $O(m * (n + m))$

```

#include <iostream>
#include <vector>
#include <queue>
#include <string>
#include <cstring>
using namespace std;
vector<int> e[1000],tp;
int din[1000];
int vis[1000];
int n,m;
int ans=0;
bool toposort(int r){
    queue<int> Q;
    int t[1000];
    bool flag=true;
    for(int i = 1; i <=n; i++) {
        t[i] = din[i];
        if(!din[i]) Q.push(i), vis[i] = true;
    }
    while(!Q.empty()){
        if(Q.size()>1) flag=false;//如果同时出现两个，说明这两个要么是不能确定，要么是矛盾，
        int x=Q.front();
        Q.pop();
        tp.push_back(x);
        for(int i=0;i<e[x].size();i++){
            int it=e[x][i];
            t[it]--;
            if(!t[it]&&!vis[it]) Q.push(it),vis[it]=1;;
        }
    }
    // for(auto it:tp) cout<<char(it + 'A'-1);
    if(tp.size()<n) return false;
    if(flag && !ans) ans = r;
    return true;
}
string s[1000];
int main(){
    while(cin>>n>>m&&n+m!=0){
        bool flg=0;
        tp.clear();
        for(int i=1;i<=n;i++){
            e[i].clear();
            din[i]=0;

```

```

}
ans=0;
memset(vis,0,sizeof(vis));
for(int i=1;i<=m;i++){
    cin>>s[i];
}
for(int i = 1; i <= m; i++) {
    string c;
    c=s[i];
    int x = c[0] - 'A'+1, y = c[2] - 'A'+1;
    e[x].push_back(y);
    din[y]++;
    if(!toposort(i)) {
        cout<<"Inconsistency found after "<<i<<" relations."<<endl;
        // return 0;
        flg=1;
        break;
    }
    if(ans){
        cout<<"Sorted sequence determined after "<<ans<<" relations: ";
        for(int i=0;i<tp.size();i++) cout<<char(tp[i] + 'A'-1);
        cout<<"."<<endl;
        // return 0;
        flg=1;
        break;
    }
    memset(vis, 0, sizeof(vis));
    if(i!=m) tp.clear();
}
if(!flg){
    if(ans){
        cout<<"Sorted sequence determined after "<<ans<<" relations: ";
        for(int i=0;i<tp.size();i++) cout<<char(tp[i] + 'A'-1);
        cout<<"."<<endl;
    }else cout<<"Sorted sequence cannot be determined."<<endl;
}
}
return 0;
}

```


第四次上机实验

第二题

- 最小生成树板子题，时间复杂度 $O(n\log n)$

```

#include <iostream>
#include <algorithm>
using namespace std;
#define int long long
#define inf 0x3f3f3f3f3f3f3f3f
#define pii pair<int, int>
#define endl '\n'
const int N=1e6+10;
const int N1=1e3+10;
const int mod=1e9+7;
struct vec{
    int x,y,z;
    bool operator<(const vec &other)const{
        return z<other.z;
    }
}a[510000];
int ec=0;
int n,m;
int f[510000];
int find_f(int x){
    if(f[x]==x) return x;
    return f[x]=find_f(f[x]);
}
void join(int x,int y){
    int fx=find_f(x);
    int fy=find_f(y);
    if(fx!=fy){
        f[fx]=fy;
    }
}
int krusal(){//每次将权最小的边连入并查集，判断是否有环，同时如果连入边的个数小于n-1，说明不连通。
    int cnt=0,ans=0;
    sort(a+1,a+1+ec);
    for(int i=1;i<=n;i++){
        f[i]=i;
    }
    for(int i=1;i<=ec;i++){
        if(find_f(a[i].x)==find_f(a[i].y)) continue;
        else{
            cnt++,ans+=a[i].z;
            join(a[i].x,a[i].y);
        }
    }
}

```

```

        return ans;
    }
    signed main(){
        // ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);
        while(cin>>n&& n){
            ec=0;
            for(int i=1;i<=n-1;i++){
                int nw=-1,to=-1,w=0;
                char x;cin>>x;
                int p;cin>>p;
                for(int j=1;j<=p;j++){
                    char y;cin>>y;
                    int w;cin>>w;
                    a[++ec]=vec{x-'A'+1,y-'A'+1,w};
                }
            }
            cout<<krusal()<<endl;
        }
        return 0;
    }
}

```

第三题

- 找欧拉回路，思路是找出每个环
- 核心代码：每次把一个环都走完
- 时间复杂度 $O(m)$

```

void dfs(int u){
    while(sz[u]<e[u].size()){
        sz[u]++;
        dfs(e[u][sz[u]-1]);
    }
    ans.push(u);
}

```

```

#include <iostream>
#include <stack>
#include <algorithm>
#include <vector>
using namespace std;
// #define int long long
#define inf 0x3f3f3f3f3f3f3f3f
#define pii pair<int, int>
#define endl '\n'
#pragma comment(linker, "/STACK:64000000,64000000")
const int N=1e4+10;
const int N1=1e3+10;
const int mod=1e9+7;
vector<int> e[N];
stack<int> ans;
int sz[N];
bool flag=0;
void dfs(int u){
    while(sz[u]<e[u].size()){
        sz[u]++;
        dfs(e[u][sz[u]-1]);
    }
    ans.push(u);
}
void Atomic_AC_machine(){
    int n,m;cin>>n>>m;
    for(int i=1;i<=m;i++){
        int u,v;cin>>u>>v;
        e[u].push_back(v);
        e[v].push_back(u);
    }
    dfs(1);
    while(ans.size()){
        cout<<ans.top()<<endl;
        ans.pop();
    }
}
signed main(){
    // ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);
    int t=1;
    // cin>>t;
    while(t--){
        Atomic_AC_machine();
    }
}

```

```
    }  
    return 0;  
}
```

第四题：

- 注意到如果能走的话一定是擦边走，所以题目中的关键节点一共不超过几个，把所有能互相到达的点连边跑最短路就行了。
- 连边过程中需要判断线段和线段是否相交，这里当与端点重合的时候可以算不相交，不影响。

```

#include <iostream>
#include <cmath>
#include <vector>
#include <iomanip>
using namespace std;
#define int long long
#define inf 0x3f3f3f3f3f3f3f3f
#define pii pair<int, int>
#define endl '\n'
const int N=1e6+10;
const int N1=1e3+10;
const int mod=1e9+7;
#define db double
const db eps = 1e-8;          // 根据题目精度要求进行修改
const db PI = acos(-1.0);    // pai, 3.1415916....

db d[N1][N1];
int sgn(db x) { // 进行判断, 提高精度
    if (fabs(x) < eps)
        return 0;          // x == 0, 精度范围内的近似相等
    return x > 0 ? 1 : -1;  // 返回正负
}
typedef struct Point {
    db x, y;
    Point(db x = 0, db y = 0) : x(x), y(y) {} // 构造函数, 初始值为 0

    // 重载运算符
    // 点 - 点 = 向量(向量AB = B - A)
    Point operator-(const Point& B) const { return Point(x - B.x, y - B.y); }

    // 点 + 点 = 点, 点 + 向量 = 向量, 向量 + 向量 = 向量
    Point operator+(const Point& B) const { return Point(x + B.x, y + B.y); }

    // 向量 × 向量 (叉积)
    db operator^(const Point& B) const { return x * B.y - y * B.x; }

    // 向量 · 向量 (点积)
    db operator*(const Point& B) const { return x * B.x + y * B.y; }

    // 点 * 数 = 点, 向量 * 数 = 向量
    Point operator*(const db& B) const { return Point(x * B, y * B); }

    // 点 / 数 = 点, 向量 / 数 = 向量

```

```

Point operator/(const db& B) const { return Point(x / B, y / B); }

// 判断大小，一般用于排序
bool operator<(const Point& B) const {
    return x < B.x || (x == B.x && y < B.y);
}

// 判断相等，点 == 点，向量 == 向量，一般用于判断和去重
bool operator==(const Point& B) const {
    return sgn(x - B.x) == 0 && sgn(y - B.y) == 0;
}

// 判断不相等，点 != 点，向量 != 向量
bool operator!=(const Point& B) const {
    return sgn(x - B.x) || sgn(y - B.y);
}
}Vector;

int Cross(Point a, Point b, Point c) {
    return sgn((b - a) ^ (c - a));
}

bool OnSegment(Point P, Point A, Point B) {
    Vector PA = A - P, PB = B - P;
    return sgn(PA ^ PB) == 0 &&
        sgn(PA * PB) <= 0; // <=, 包括端点; <, 不包括端点
}

struct Seg {
    Point s, e;
    Seg() {}
    Seg(Point x, Point y) : s(x), e(y) {}
};

bool Intersect_seg(Point a, Point b, Point c, Point d) {
    // 第一个if是判断是否在端点处相交
    // if (OnSegment(a, c, d) || OnSegment(b, c, d) || OnSegment(c, a, b) ||
    //     OnSegment(d, a, b))
    //     return 1;
    if (Cross(a, b, c) * Cross(a, b, d) >= 0)
        return 0;
    if (Cross(c, d, a) * Cross(c, d, b) >= 0)
        return 0;
    return 1;
}

db dist(Point a, Point b) {
    return sqrt((a - b) * (a - b));
}

```

```

}
signed main(){
    ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);
    int n;
    while(cin>>n&&n!=-1){
        vector<Point> pi;
        vector<Seg> sg;
        pi.push_back(Point(0,5));
        for(int i=1;i<=n;i++){
            db x;cin>>x;
            db y1,y2,y3,y4;
            cin>>y1>>y2>>y3>>y4;
            Point O(x,0);
            Point a(x,y1);
            Point b(x,y2);
            Point c(x,y3);
            Point d(x,y4);
            Point E(x,10);
            pi.push_back(a);
            pi.push_back(b);
            pi.push_back(c);
            pi.push_back(d);
            sg.push_back(Seg(0,a));
            sg.push_back(Seg(b,c));
            sg.push_back(Seg(d,E));
        }
        pi.push_back(Point(10,5));
        for(int i=0;i<pi.size();i++){
            for(int j=i;j<pi.size();j++){
                if(i==j) {
                    d[i+1][j+1]=d[j+1][i+1]=0;
                    continue;
                }
                Seg nw(pi[i],pi[j]);
                bool check=0;
                for(int k=0;k<sg.size();k++){
                    if(Intersect_seg(nw.e,nw.s,sg[k].e,sg[k].s)){
                        // cout<<i<<" "<<j<<" "<<k<<endl;
                        check=1;
                        break;
                    }
                }
            }
            if(!check){

```



```

        // cout<<i<<" "<<j<<endl;
        d[j+1][i+1]=d[i+1][j+1]=dist(pi[i],pi[j]);
    }else d[j+1][i+1]=d[i+1][j+1]=inf;
}
}
int len=pi.size();
for(int k=1;k<=len;k++){
    for(int i=1;i<=len;i++){
        for(int j=1;j<=len;j++){
            d[i][j]=min(d[i][k]+d[k][j],d[i][j]);
        }
    }
}
cout<<fixed<<setprecision(2)<<d[1][len]<<endl;
}
return 0;
}

```

第五题

- 选择一个起始点，然后以这个起始点为基准进行极角排序，每次走偏左的就行，可以保证n个点都能走到。
- 时间复杂度 $O(n^2 \log n)$ 。

```

#include <iostream>
#include <cmath>
#include <algorithm>
#include <vector>
using namespace std;
#define int long long
#define inf 0x3f3f3f3f3f3f3f3f
#define pii pair<int, int>
#define endl '\n'
const int N=1e3+10;
const int N1=1e3+10;
const int mod=1e9+7;
#define db long double
struct Point {
    db x, y,id;
    Point(db id = 0,db x = 0, db y = 0) : id(id), x(x), y(y) {} // 构造函数, 初始值为 0
}p[N];
int pos; //基准点
int cross(Point p0, Point p1, Point p2) {
    return (p1.x - p0.x) * (p2.y - p0.y) - (p2.x - p0.x) * (p1.y - p0.y);
}
int dis(Point p1, Point p2) {
    return (p2.x - p1.x) * (p2.x - p1.x) + (p2.y - p1.y) * (p2.y - p1.y);
}
bool cmp(Point p1, Point p2) {
    int tmp = cross(p[pos], p1, p2);
    if (tmp > 0) return true;
    else if (tmp == 0 && dis(p[pos], p1) < dis(p[pos], p2)) return true;
    else return false;
}
void Atomic_AC_machine(){
    int n;cin>>n;
    vector<int> ans;
    for(int i=1;i<=n;i++){
        int id,x,y;
        cin>>id>>x>>y;
        p[i]=Point(id,x,y);
        if(p[i].y<p[1].y) swap(p[i],p[1]);
    }
    ans.push_back(p[1].id);
    cout<<n<<" ";
    pos=1;
    for(int i=2;i<=n;i++){

```

```

        sort(p+i,p+1+n,cmp);
        ans.push_back(p[i].id);
        pos=i;
    }
    for(int i=0;i<ans.size();i++){
        cout<<ans[i]<<" ";
    }
    cout<<endl;
}
signed main(){
    ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);
    int t=1;
    cin>>t;
    while(t--){
        Atomic_AC_machine();
    }
    return 0;
}

```

第六题：

- 二进制枚举砍的树，求剩下树的凸包，看看能不能围上
- 时间复杂度 ($2^n n \log n$)

```

// 凸包
// Andrew算法求凸包，最后一个点与第一个点重合
// Need: (^,-,<), sgn, le
#include <iostream>
#include <cmath>
#include <algorithm>
#include <vector>
#include <iomanip>
using namespace std;
#define int long long
#define inf 0x3f3f3f3f3f3f3f3f
#define pii pair<int, int>
#define endl '\n'
const int N=1e6+10;
const int N1=1e3+10;
const int mod=1e9+7;
#define db double
const db eps = 1e-8;          // 根据题目精度要求进行修改
const db PI = acos(-1.0);    // pai, 3.1415916...
int sgn(db x) { // 进行判断，提高精度
    if (fabs(x) < eps)
        return 0;          // x == 0, 精度范围内的近似相等
    return x > 0 ? 1 : -1;  // 返回正负
}
bool eq(db a, db b) {
    return abs(a - b) < eps;
} // ==
bool gt(db a, db b) {
    return a - b > eps;
} // >
bool lt(db a, db b) {
    return a - b < -eps;
} // <
bool ge(db a, db b) {
    return a - b > -eps;
} // >=
bool le(db a, db b) {
    return a - b < eps;
} // <=
typedef struct Point {
    db x, y,v,l,id;
    Point(db x = 0, db y = 0 , db v = 0 , db l = 0 , db id = 0) : x(x), y(y), v(v), l(l), id(id)

```

```

// 重载运算符
// 点 - 点 = 向量(向量AB = B - A)
Point operator-(const Point& B) const { return Point(x - B.x, y - B.y); }

// 点 + 点 = 点, 点 + 向量 = 向量, 向量 + 向量 = 向量
Point operator+(const Point& B) const { return Point(x + B.x, y + B.y); }

// 向量 × 向量 (叉积)
db operator^(const Point& B) const { return x * B.y - y * B.x; }

// 向量 · 向量 (点积)
db operator*(const Point& B) const { return x * B.x + y * B.y; }

// 点 * 数 = 点, 向量 * 数 = 向量
Point operator*(const db& B) const { return Point(x * B, y * B); }

// 点 / 数 = 点, 向量 / 数 = 向量
Point operator/(const db& B) const { return Point(x / B, y / B); }

// 判断大小, 一般用于排序
bool operator<(const Point& B) const {
    return x < B.x || (x == B.x && y < B.y);
}

// 判断相等, 点 == 点, 向量 == 向量, 一般用于判断和去重
bool operator==(const Point& B) const {
    return sgn(x - B.x) == 0 && sgn(y - B.y) == 0;
}

// 判断不相等, 点 != 点, 向量 != 向量
bool operator!=(const Point& B) const {
    return sgn(x - B.x) || sgn(y - B.y);
}
} Vector;

db dist(Point a, Point b) {
    return sqrt((a - b) * (a - b));
}

bool check(Point p, Point q, Point r) {
    return le(0, (q - p) ^ (r - q));
}

db Andrew(vector<Point>& poly) { // 末尾额外塞了个头
    int n = poly.size(), top = 0;
    vector<int> stk(n + 2, 0), used(n + 2, 0);

```

```

sort(poly.begin(), poly.end());
stk[++top] = 0;
for (int i = 1; i < n; i++) {
    while (top > 1 &&
           sgn((poly[stk[top]] - poly[stk[top - 1]]) ^
               (poly[i] - poly[stk[top]])) <= 0) // 去掉等号可以找共线的点
        used[stk[top--]] = 0;

    used[i] = 1;
    stk[++top] = i;
}
int tmp = top;
for (int i = n - 2; i >= 0; i--) {
    if (used[i])
        continue;
    while (top > tmp && sgn((poly[stk[top]] - poly[stk[top - 1]]) ^
                           (poly[i] - poly[stk[top]])) <= 0)
        used[stk[top--]] = 0;

    used[i] = 1;
    stk[++top] = i;
}
vector<Point> a;
for (int i = 1; i <= top; i++)
    a.push_back(poly[stk[i]]);
db sum=0;
for(int i=1;i<a.size();i++){
    sum+=dist(a[i],a[i-1]);
}
sum+=dist(a[0],a.back());
return sum;
}
bool cmp1(Point x,Point y){
    return x.id<y.id;
}
signed main(){
    int n;
    int id=0;
    ios::sync_with_stdio(false);
    cin.tie(0);cout.tie(0);
    int flag=0;
    while(cin>>n&&n){
        if(flag++) cout<<endl;

```

```

vector<Point> pi;
for(int i=1;i<=n;i++){
    int x,y,v,l;
    cin>>x>>y>>v>>l;
    pi.push_back(Point(x,y,v,l,i));
}
db ans=inf;
db anscost=inf;
vector<Point> ansP;
for(int i=0;i<(1<<n);i++){
    vector<Point> poly;
    vector<Point> rest;
    db hv=0,h1=0;
    for(int j=0;j<n;j++){
        if(i&(1<<j)){
        }else{
            hv+=pi[j].v;
            h1+=pi[j].l;
        }
    }
    if(hv-anscost>eps){
        continue;
    }
    for(int j=0;j<n;j++){
        if(i&(1<<j)){
            poly.push_back(pi[j]);
        }
    }
    if(!poly.size()) continue;
    db cost=Andrew(poly);
    if(h1-cost>eps&&anscost-hv>eps){
        for(int j=0;j<n;j++){
            if(!(i&(1<<j))){
                rest.push_back(pi[j]);
            }
        }
        ans=h1-cost;
        anscost=hv;
        ansP=rest;
    }else if(h1-cost>eps&&anscost-hv<=eps){
        for(int j=0;j<n;j++){
            if(!(i&(1<<j))){
                rest.push_back(pi[j]);
            }
        }
    }
}

```

```

        }
    }
    if(ansP.size() > rest.size()) ansP = rest;
}
}
id++;
cout<<"Forest " <<(int)id<<endl;
cout<<"Cut these trees:";
sort(ansP.begin(), ansP.end(), cmp1);
for(int i=0; i<ansP.size(); i++){
    cout<<" " <<(int)ansP[i].id;
}
cout<<endl;
cout<<fixed<<setprecision(2)<<"Extra wood: " <<ans<<endl;
}
}

```